🗓️ Day 1 – Project Setup + Auth UI

Goal: Get frontend + backend ready, design Login/Signup, and test routes.

🧩 Frontend (React)

- Create React app (Vite preferred).
- Set up file structure:

```
src/
 ├── pages/
 ├── components/
 ├── redux/
 ├── api/
```

- Install packages:
  `npm install react-router-dom axios redux @reduxjs/toolkit react-redux`
- Create routes: `/login`, `/signup`, `/dashboard`
- Build UI for Login + Signup pages (basic form + validation).

⚙️ Backend (Express + MongoDB)

- Setup Express server, connect MongoDB.
- Create `User` model → `{ name, email, password }`
- Implement `signup` & `login` routes with password hashing (bcrypt).
- Return JWT token on login.
- Test in Postman.

✅ **End of Day:** You can create an account & login → token returns in console.

🗓️ Day 2 – Redux Toolkit (Login + Token Management)

Goal: Learn Redux Toolkit through actual login/logout logic.

🧠 Learn (practically)

- Create `authSlice.js` → store user token + details.
- Create `store.js` and wrap app with `<Provider>`.
- Implement:
  - `loginSuccess`
  - `logout`
  - `loadUserFromStorage`
- On login success, save token to `localStorage` & Redux store.
- On logout, clear token.
- Protect routes → if not logged in, redirect to `/login`.

✅ **End of Day:** Login works → token persists via Redux. Logout clears session.

🗓️ Day 3 – Dashboard + CRUD Setup

Goal: Build dashboard layout & backend for ideas/posts.

⚙️ Backend

- Create `Idea` model → `{ title, platform, caption, image, status, userId }`.
- Create controller + router:
  - `POST /ideas` → Add new
  - `GET /ideas` → Fetch all for logged-in user
  - `PUT /ideas/:id` → Update
  - `DELETE /ideas/:id` → Delete
- Use JWT middleware to validate user.

🧩 Frontend

- Create Dashboard page:
  - Show all ideas in a grid/list.
  - Button ➕ to open Add Idea form (modal or new page).
- Use Axios to call `/ideas` APIs with Bearer token.
- Store ideas in Redux (new slice: `ideaSlice.js`).

✅ **End of Day:** You can view all ideas from MongoDB, stored per user.

🗓️ Day 4 – Add / Edit / Delete with Image Upload

Goal: Implement CRUD + image upload (Multer + Cloudinary).

⚙️ Backend

- Add Multer + Cloudinary logic inside `POST` and `PUT` routes for image field.
- Store only Cloudinary `secure_url` in MongoDB.

🧩 Frontend

- Add form for idea:
  - Title
  - Platform (select)
  - Caption
  - Image upload
- On submit → send `FormData` with Axios to backend.
- Add Edit/Delete buttons on dashboard cards.

✅ **End of Day:** Full CRUD works with images saved on Cloudinary 🥕

🗓️ Day 5 – UI Polish + Analytics + Final Touch

Goal: Make it beautiful and professional.

🎨 Frontend Polish

- Add analytics (count total, posted, drafts).
- Add dark/light mode toggle (optional).
- Add loader & success/error messages.
- Animate modals with Framer Motion.
- Deploy frontend → Vercel, backend → Render.

✅ **End of Day:** You have a production-ready project with login, Redux auth, CRUD, image uploads & analytics.